

Empirical Evaluation of Agentic Architectures for Scientific Discovery

Materials Informatics and Cheminformatics

Alexander Girutskiy
Eötvös Lóránd University (ELTE)

May 2026

Abstract

Large language models (LLMs) are increasingly used as agents that plan, call external tools, validate intermediate evidence, and collaborate with other specialized agents. This project evaluates whether increasing agentic complexity improves reliability in scientific discovery workflows. I implemented a reproducible LangGraph-based pipeline comparing four architectures: a Level 1 single-agent baseline, a Level 2A Planner-Executor system, a Level 2B Solver-Critic system, and a Level 3 adaptive memory multi-agent system. The systems were evaluated across two API-driven scientific domains, materials informatics and cheminformatics, each with an easy retrieval task and a harder multi-hop screening task. All evaluations used deterministic, rule-grounded checks rather than subjective LLM-as-judge scoring. Easy tasks saturated at a score of 1.0 for all architectures. Hard tasks revealed differences: the single-agent baseline achieved a mean score of 0.937, Planner-Executor achieved 0.959, and both Solver-Critic and Adaptive Memory achieved 0.977. The best accuracy-efficiency trade-off was Solver-Critic: it matched Level 3 in mean score and strict pass rate while using fewer steps and fewer effective tokens. The main conclusion is that reflection and verification improve reliability on multi-hop scientific tasks, but additional adaptive memory adds overhead and did not improve accuracy in this two-run experiment.

1 Introduction

Scientific uses of LLMs require more than fluent text generation. In materials informatics, an assistant may need to query computed materials databases, parse structural descriptions, compare band gaps and thermodynamic stability, and justify why a candidate satisfies explicit scientific constraints. In cheminformatics, an assistant may need to retrieve compound properties, resolve biological targets, aggregate activity measurements, compare off-target evidence, and rank molecules according to transparent rules. These tasks require planning, tool use, evidence checking, and failure analysis.

This report studies the following research question:

When do multi-agent and adaptive agent architectures outperform a minimal single-agent baseline in scientific information-seeking tasks?

The project follows the assigned structure: at least three architectures of increasing agentic complexity, two task domains, two difficulty levels, controlled comparisons, multiple runs, quantitative metrics, efficiency analysis, and a Python-based agentic framework. The implementation uses LangGraph because it is designed as a low-level orchestration framework for long-running, stateful agents, with explicit graph state and execution transitions [2]. The experiment is empirical rather than demonstrative: the notebook exports raw and aggregated CSV files, and the report is derived from those outputs.

I tested three hypotheses. First, the single-agent baseline should be sufficient for easy retrieval tasks because the output contract is short and the relevant evidence is directly available from one endpoint. Second, Level 2 multi-agent systems should improve hard-task reliability because planning, execution, and criticism are separated into different roles. Third, the Level 3 adaptive memory architecture should be most reliable but most expensive because it adds memory recall and memory writing to the Solver-Critic pattern.

2 Related Work

The project is motivated by tool-using LLM agents. ReAct showed that interleaving reasoning traces with actions can improve interpretability and allow models to interact with external environments or knowledge sources [1]. In chemistry, ChemCrow connected an LLM to eighteen expert-designed chemistry tools and demonstrated that tool integration can extend LLM performance in organic synthesis, drug discovery, and materials design [9]. SciAgents extended the idea to a multi-agent scientific-discovery setting, combining LLMs, retrieval tools, and knowledge-graph reasoning for biologically inspired materials hypotheses [10].

LangGraph is used as the framework layer. Its documentation describes it as an orchestration runtime for durable, stateful workflows and agents, with support for memory, persistence, state transitions, and complex control flow [2]. This is important for the present experiment because the same task objects can be routed through different graph structures while preserving a consistent evaluation pipeline.

The materials domain uses the Materials Project API. The Materials Project documentation provides API examples for summary queries, structures, band gaps, phase diagrams, and Robocrystallographer-derived searches [3]. The hard materials task specifically uses a documented challenge: structure dimensionality is not stored as a formal summary field, so entries must be categorized by parsing Robocrystallographer analyses; the documentation also states that material documents and Robocrystallographer documents are retrieved with separate queries and manually associated [4].

The cheminformatics domain uses PubChem and ChEMBL. PubChem PUG REST is used for the easy aspirin property lookup. ChEMBL Data Web Services are used for target search, molecule retrieval, activity retrieval, filtering, pagination, and JSON outputs. ChEMBL documents JSON output formats, page metadata, filters such as molecular-weight and AlogP constraints, and activity queries using fields such as `target_chembl_id`, `pchembl_value_gte`, and `only` [5, 6, 7].

3 Methodology

3.1 Task domains and difficulty levels

The experiment contains two domains. The materials-informatics domain uses the Materials Project API to retrieve computed inorganic materials data. The cheminformatics domain uses PubChem and ChEMBL to retrieve molecular properties and target-activity data. These domains were selected because both are structured scientific information-seeking domains with public APIs, but their hard tasks stress different weaknesses. The materials hard task emphasizes structural description parsing and sparse polymorph evidence. The cheminformatics hard task emphasizes multi-hop target resolution, activity aggregation, off-target comparison, and ranking.

Table 1: Task design. Easy tasks test direct retrieval; hard tasks test multi-hop planning, evidence aggregation, and rule-grounded ranking.

Domain	Difficulty	Task ID	Main requirements
Materials	Easy	<code>materials_easy_batio3</code>	Retrieve one BaTiO3 row with band gap and energy above hull from Materials Project.
Materials	Hard	<code>materials_hard_dimensional_polymorphs</code>	Search candidate chemical systems; retrieve summary and Robocrystallographer data; classify low-dimensional candidates; compare stability, band gap, structures, 3D same-formula evidence, ranking scores, and rejected candidates.
Cheminformatics	Easy	<code>chem_easy_aspirin</code>	Retrieve aspirin formula, molecular weight, SMILES, XLogP, and hydrogen-bond counts from PubChem.
Cheminformatics	Hard	<code>chem_hard_selective_egfr_screen</code>	Resolve EGFR and off-targets in ChEMBL; retrieve IC50 records; aggregate pChEMBL values; require multiple primary measurements; compute selectivity margins; apply drug-like filters; return three ranked candidates.

Difficulty is increased in three ways. First, the hard tasks have a longer planning horizon: they require search, filtering, aggregation, ranking, and validation. Second, they include partial observability and missing evidence: a candidate may lack a Robocrystallographer dimensionality signal or may have incomplete off-target activity

records. Third, they use strict output contracts: a scientifically plausible result can still fail if it omits structures, returns too few molecules, or fails to establish a same-formula 3D polymorph.

3.2 Architectures

Four architectures were evaluated. They differ only in graph structure and agent roles; the task definitions, tool functions, and deterministic validators are shared.

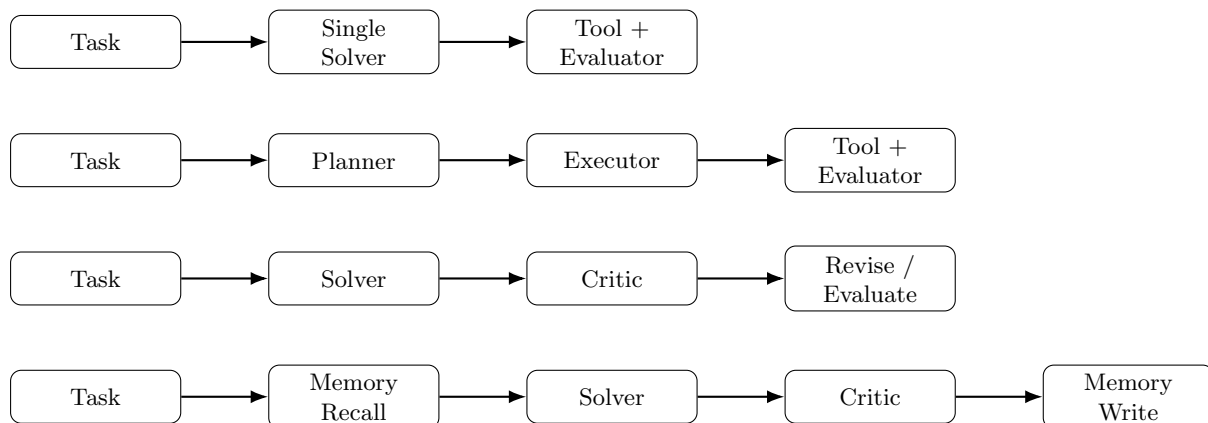


Figure 1: Implemented architecture patterns. Level 1 performs one LLM planning call. Level 2A separates planning from executable-plan construction. Level 2B adds an LLM critic and revision path. Level 3 adds memory recall and memory writing around the Solver–Critic loop.

Level 1: Single-Agent Baseline. A single LLM agent converts the task into an executable JSON plan and then the deterministic tool layer executes it. There is no explicit planning module, no separate critic, and no memory.

Level 2A: Planner–Executor. The planner agent produces a strategy and constraint list. The executor agent converts that strategy into the strict executable tool plan. This produces two LLM calls per task and tests whether explicit decomposition improves reliability.

Level 2B: Solver–Critic. The solver creates a plan and result, and the critic reviews the output against the task requirements. If the critic requests revision, the solver can run a revised attempt. This creates more LLM calls than Planner–Executor but is intended to catch missing evidence and output-contract failures.

Level 3: Adaptive Memory Multi-Agent. This architecture adds memory recall before solving and memory writing after evaluation. It is expected to be the most expensive but potentially the most robust if prior failures are reusable.

3.3 Evaluation

Every task is evaluated by deterministic checks, not by subjective LLM judgement. A score is the fraction of checks passed, and `passed=True` requires a score of exactly 1.0. This distinction matters: a hard-task score of 0.909 means most scientific requirements were satisfied, not that the workflow crashed. Efficiency metrics include runtime, graph steps, LLM calls, and token usage. Token efficiency was measured using API-reported `usage` metadata when available; for OpenAI-compatible endpoints that omit usage, the notebook also records estimated token counts as a fallback. The primary token column is `effective_total_tokens`, which equals `actual_total_tokens` when present and `estimated_total_tokens` otherwise.

4 Experiments

The experiment uses a full factorial design: four architectures $\times$ two domains $\times$ two difficulty levels $\times$ two runs, producing 32 raw observations. The notebook completed all cells without execution errors. The final validity checks showed that all rows used the LLM and that no row contained LLM, API, or JSON-parser errors.

Table 2: Execution validity summary.

Metric	Value
Raw experiment rows	32
Rows with <code>llm_used=True</code>	32
Rows with hidden API/LLM/parser errors	0
Rows with strict pass	20
Strict overall pass rate	0.625
Repeated runs per architecture-task condition	2
API usage reported (<code>actual_usage_available_rate</code>)	1.0

5 Results

5.1 Score matrix

Table 3 gives the central architecture-by-task comparison. The easy tasks saturate: every architecture scores 1.0 on both direct-retrieval tasks. The hard tasks are more informative. The single-agent baseline loses points on both hard tasks. Planner-Executor improves materials hard but still returns only two molecules on the ChEMBL selectivity task. Solver-Critic and Adaptive Memory both solve the ChEMBL hard task completely and achieve the best mean score overall.

Table 3: Mean score matrix by architecture, domain, and difficulty.

Architecture	Chem Easy	Chem Hard	Materials Easy	Materials Hard
Level 1: Single-Agent Baseline	1.000	0.929	1.000	0.818
Level 2A: Planner + Executor	1.000	0.929	1.000	0.909
Level 2B: Solver + Critic	1.000	1.000	1.000	0.909
Level 3: Adaptive Memory Multi-Agent	1.000	1.000	1.000	0.909

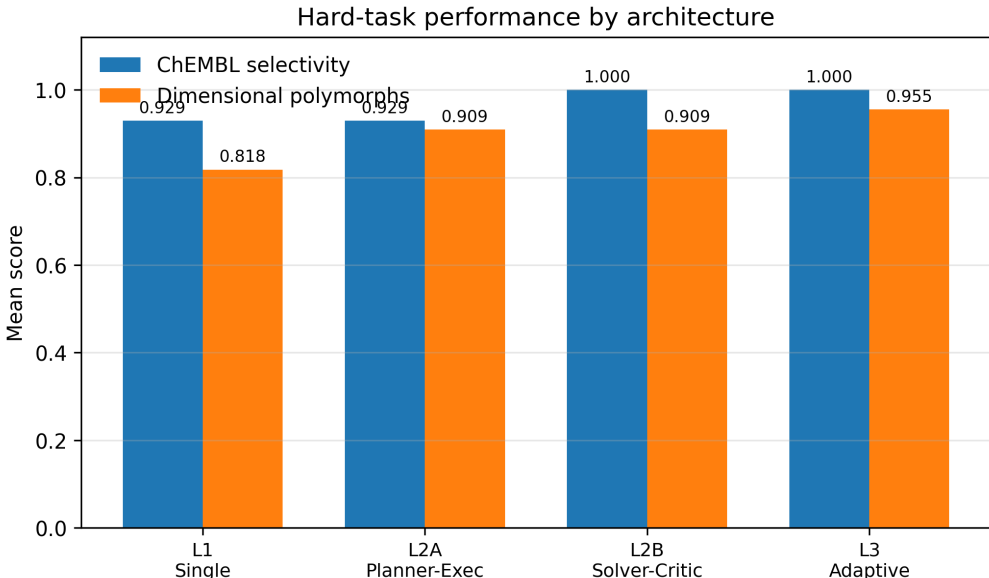


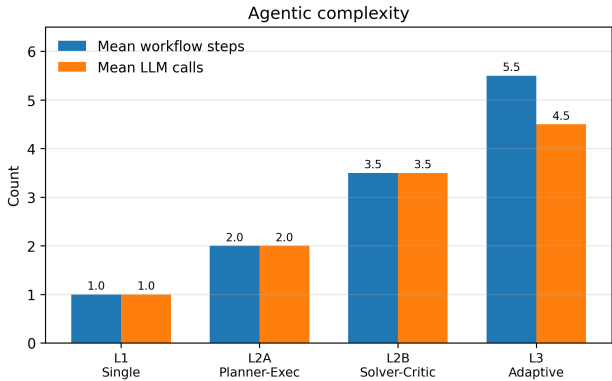
Figure 2: Hard-task performance by architecture. The ChEMBL selectivity task separates the reflection-based architectures from the simpler systems, while the materials dimensional-polymorph task remains difficult for all architectures under the strict same-formula 3D-pair criterion.

5.2 Architecture-level efficiency

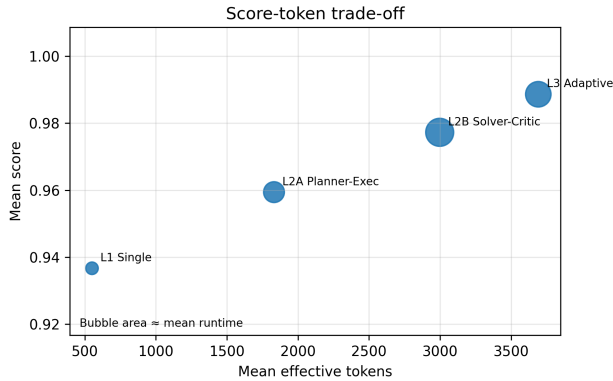
Table 4 shows that the performance gains are not free. Level 1 is fastest and cheapest, with one step and one LLM call. Level 2A doubles the LLM calls and improves mean score from 0.937 to 0.959. Level 2B raises mean score to 0.977 and strict pass rate to 0.75, but increases calls and tokens. Level 3 matches Level 2B in score and pass rate but uses the most steps and tokens.

Table 4: Architecture-level efficiency summary. Effective tokens are mean `effective_total_tokens`.

Architecture	Mean score	Pass rate	Runtime (s)	Steps	LLM calls	Effective tokens
Level 1: Single-Agent Baseline	0.937	0.500	8.46	1.0	1.0	441.5
Level 2A: Planner + Executor	0.959	0.500	14.22	2.0	2.0	1804.0
Level 2B: Solver + Critic	0.977	0.750	20.08	3.5	3.5	2350.0
Level 3: Adaptive Memory Multi-Agent	0.977	0.750	17.75	5.5	4.5	3058.1



(a) Agentic complexity.



(b) Score-token trade-off.

Figure 3: Increasing agentic complexity increases workflow steps, LLM calls, and effective token usage. Solver-Critic provides the best observed trade-off because it matches Level 3 accuracy with lower overhead.

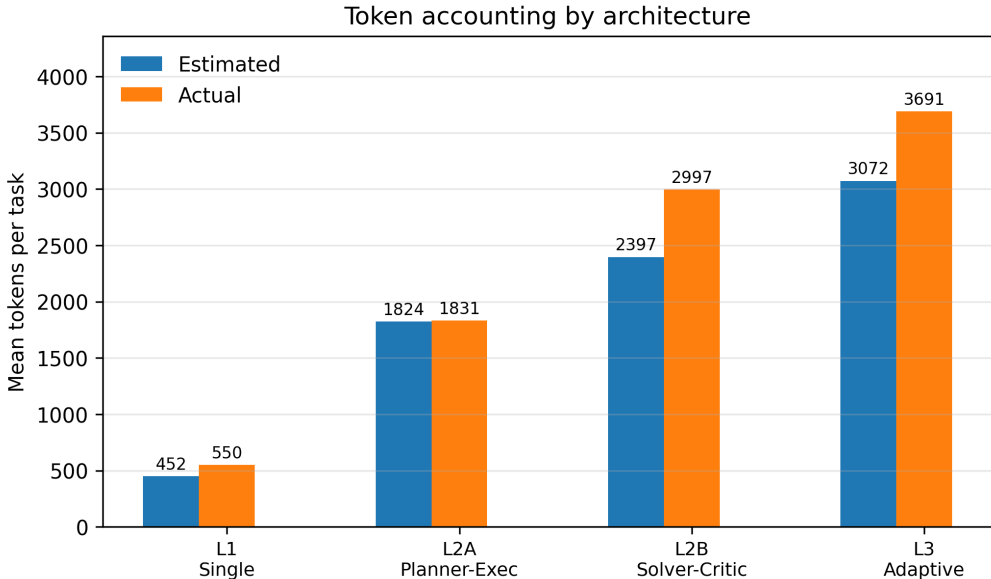


Figure 4: Token accounting by architecture. Estimated totals are text-length fallbacks; actual totals come from `API usage` metadata when available; effective totals are the primary cost metric used in the efficiency analysis.

5.3 Task-level behavior

Aggregating by domain and difficulty shows the intended difficulty gradient. Easy tasks have perfect mean scores and pass rates. Hard cheminformatics has a high mean score but only half of conditions strictly pass because Level 1 and Level 2A return two candidates instead of the required three. Hard materials has a high partial score but a strict pass rate of zero because all architectures miss at least one stringent requirement.

Table 5: Task-family summary averaged over architectures.

Domain	Difficulty	Mean score	Mean strict pass rate	Mean runtime (s)
Materials	Easy	1.000	1.000	10.27
Materials	Hard	0.886	0.000	33.60
Cheminformatics	Easy	1.000	1.000	3.25
Cheminformatics	Hard	0.964	0.500	13.40

The final artifacts show representative candidates. For the materials hard task, the best Level 3 candidate in the first run is **WS2** with band gap 1.5657 eV, energy above hull 0.0, a two-dimensional Robocrystallographer-derived label, and ranking score 0.8397. This is a strong low-dimensional semiconductor candidate, but it fails the strict same-formula 3D polymorph-pair check. For the ChEMBL hard task, the Level 3 result returns three EGFR-selective molecules including **CHEMBL29197**, **CHEMBL3623290**, and **CHEMBL5957231**, with primary median pChEMBL values above 9.7 and selectivity margins above 2.3.

5.4 Failure checks

Table 6: Failed validation checks across the final run. Counts are over all architecture-run combinations.

Task group	Failed check	Count
Materials hard	same_formula_3d_polymorph_found	6
Materials hard	structures_retrieved	4
Cheminformatics hard	returned_three_candidates	4

The failure table reveals three distinct failure modes. First, the materials task often finds plausible low-dimensional candidates but cannot establish a same-formula 3D polymorph. Second, the simpler materials agents sometimes fail to return structure summaries for all candidates. Third, the simpler cheminformatics agents find valid EGFR-selective candidates but stop at two molecules rather than the required three.

6 Analysis

6.1 Hypothesis evaluation

The first hypothesis is supported. Easy retrieval tasks are solved by every architecture. For short, direct API lookups, adding planner, critic, or memory agents does not improve accuracy. This is important because agentic complexity has cost: extra nodes increase prompts, tokens, implementation surface area, and runtime.

The second hypothesis is partially supported. Multi-agent systems improve hard-task performance, but not uniformly. Planner-Executor improves over the single-agent baseline on the materials hard task, increasing the score from 0.818 to 0.909, but it does not solve the ChEMBL task completely. Solver-Critic performs best on hard cheminformatics by catching and revising missing-candidate failures. This suggests that reflection is particularly useful when success depends on validating an output contract rather than merely choosing the right API endpoint.

The third hypothesis is only partially supported. The adaptive memory system is reliable, but it does not outperform Solver-Critic in this small two-run benchmark. It matches the Solver-Critic mean score (0.977) and pass rate (0.75), while using more steps and effective tokens. This suggests that adaptive memory is not automatically beneficial; it becomes worthwhile only when the task distribution contains repeated, learnable failure patterns.

6.2 Why the hard materials task remained difficult

The hard materials task is a stress test. The agents successfully used Materials Project and Robocrystallographer data, retrieved low-dimensional candidates, and ranked them. However, the strict validator required a stronger claim: the selected low-dimensional material needed evidence of a same-formula 3D polymorph and associated structure information. This is difficult because dimensionality is not a normal Materials Project summary field and must be inferred from Robocrystallographer text, while materials and Robocrystallographer documents must be retrieved and joined manually [4]. Under this strict condition, the agents generally selected scientifically plausible low-dimensional semiconductors but did not fully satisfy the polymorph-pair constraint.

6.3 Why the hard ChEMBL task separated architectures

The ChEMBL task requires several linked operations: target resolution, activity retrieval, pChEMBL aggregation, off-target evidence collection, drug-like filtering, and ranking. The Level 1 and Level 2A systems completed most checks but returned only two final molecules. Solver-Critic and Adaptive Memory returned all three molecules and passed every check. This supports the value of reflection when the main failure mode is not factual retrieval but completeness of the final answer contract.

6.4 Trade-off interpretation

The best practical architecture in this experiment is Solver-Critic. It provides the same mean score and pass rate as Level 3 while using fewer steps and fewer effective tokens. Level 3 is still useful as a demonstration of adaptive architecture design, but this benchmark does not show an empirical benefit from memory. The single-agent baseline remains attractive for easy tasks because it is cheap and accurate, but it is less robust on multi-hop tasks.

7 Conclusion

This project implemented and evaluated a LangGraph-based experimental pipeline for comparing agentic architectures in scientific discovery tasks. The final run covers four architectures, two domains, two difficulty levels, two repeated runs, rule-grounded scoring, and efficiency analysis. The notebook executed successfully: all 32 rows used the LLM and no hidden API, connection, or JSON parsing errors were recorded.

The main finding is that agentic complexity matters only when the task requires multi-hop verification. Easy API retrieval tasks saturate across all architectures. On hard tasks, Planner-Executor improves over the single-agent baseline, while Solver-Critic and Adaptive Memory reach the highest mean score. However, Level 3 does not outperform Solver-Critic despite using more steps and tokens. The most defensible conclusion is therefore not that "more agents are always better", but that specialized reflection improves reliability when scientific tasks have strict evidence and completeness requirements.

Limitations

The experiment has three main limitations. First, there are only two repeated runs per condition, so the results should be interpreted as a controlled pilot study rather than a statistically broad benchmark. Second, the hard materials task is intentionally strict and sparse; a strict pass rate of zero does not mean the workflow failed, but that the same-formula 3D polymorph condition was not satisfied. Third, token accounting depends on endpoint support: the notebook records API-reported usage when available and falls back to text-length estimates otherwise. Future work should increase the number of runs, add more chemically diverse tasks, and test whether adaptive memory improves across longer sequences of related tasks.

Reproducibility Statement

The submitted notebook reads credentials from environment variables `API_ENDPOINT`, `ELTE_API_KEY`, `MODEL_NAME`, and `MATERIALS_API_KEY`. The recorded model configuration was `Qwen/Qwen3.6-27B` through an OpenAI-compatible endpoint. The notebook exports four result files:

- `agentic_architecture_results_raw.csv`
- `agentic_architecture_results_summary.csv`
- `agentic_architecture_efficiency.csv`
- `agentic_architecture_score_matrix.csv`

The tables and figures in this report were generated from the latest exported files matching `agentic_architecture_*.csv`.

References

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, J., Narasimhan, K., and Cao, Y. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629, 2022. Available at: <https://arxiv.org/abs/2210.03629>.
- [2] LangChain. *LangGraph overview*. Accessed June 2026. Available at: <https://docs.langchain.com/oss/python/langgraph/overview>.

- [3] Materials Project. *API query examples with the MPRester client*. <https://docs.materialsproject.org/downloading-data/using-the-api/examples>.
- [4] Materials Project. *Identifying Materials with Specific Structural Dimensionalities*. <https://docs.materialsproject.org/downloading-data/using-the-api/examples>.
- [5] ChEMBL. *ChEMBL Data Web Services: Supported formats and pagination*. <https://chembl.gitbook.io/chembl-interface-documentation/web-services/chembl-data-web-services>.
- [6] ChEMBL. *ChEMBL Data Web Services: Filtering and ordering*. <https://chembl.gitbook.io/chembl-interface-documentation/web-services/chembl-data-web-services>.
- [7] ChEMBL. *ChEMBL Data Web Services: Activity endpoint examples*. <https://chembl.gitbook.io/chembl-interface-documentation/web-services/chembl-data-web-services>.
- [8] NCBI PubChem. *PUG REST Documentation*. <https://pubchem.ncbi.nlm.nih.gov/docs/pug-rest>.
- [9] Bran, A. M., Cox, S., Schilter, O., Baldassari, C., White, A. D., and Schwaller, P. *ChemCrow: Augmenting large-language models with chemistry tools*. arXiv:2304.05376, 2023. <https://arxiv.org/abs/2304.05376>.
- [10] Ghafarollahi, A. and Buehler, M. J. *SciAgents: Automating scientific discovery through multi-agent intelligent graph reasoning*. arXiv:2409.05556, 2024. <https://arxiv.org/abs/2409.05556>.

Table 7: Documented use of GenAI tools during the project.

Phase	GenAI Tool Used	Validation Method
Literature exploration & planning	ChatGPT was used to brainstorm possible task domains, compare candidate APIs, and refine the experimental research question.	I manually checked that the chosen domains satisfied the project requirements: two task domains, two difficulty levels, multiple architectures, quantitative metrics, and a Python-based agentic framework. External APIs and documentation were checked directly before implementation, and the final scope was adjusted when earlier task domains proved too brittle for reliable evaluation.
Implementation	ChatGPT was used as a coding assistant to help restructure the notebook, design LangGraph workflows, define JSON-schema LLM calls, and debug API-specific issues.	All generated code was manually reviewed cell by cell. I executed the notebook locally, inspected stack traces, checked API responses, fixed missing helper functions, and verified that the final notebook completed without LLM, parser, or API errors. I also confirmed that Level 2 and Level 3 architectures used multiple LLM calls rather than only deterministic graph nodes.
Experimentation	ChatGPT was used to help interpret failed runs and identify whether failures came from the LLM, API calls, parsing, evaluation logic, or task design.	I validated the experiment outputs using the generated raw, summary, score-matrix, and efficiency CSV files. I checked that all final rows used the LLM, that no JSON parsing or API errors remained, and that the result matrix contained both easy-task saturation and meaningful hard-task differences. I did not treat failed task checks as crashes; instead, they were analyzed as part of the empirical failure analysis.
Analysis	ChatGPT was used to help summarize the quantitative results, compare architectures, and formulate the interpretation of score, pass rate, runtime, steps, LLM calls, and token usage.	I cross-checked the interpretation against the exported CSV files and the notebook outputs. Claims about architecture performance were restricted to what the metrics supported.
Report and documentation	ChatGPT was used to draft and revise sections of the LaTeX report, generate figure-creation scripts, and improve wording for the methodology, results, limitations, and reproducibility sections.	I reviewed and edited the report manually and checked that limitations were stated explicitly. I verified that the report documented the required framework, models, APIs, external libraries, task design, and evaluation metrics.